Manual Prático de Integrações v2.1 — Java Spring Boot (REST + SOAP)

Nov 9, 2026 · @SPDM - Desenvolvimento

Sumário

Atalhos mais usados: Receitas prontas · REST sem auth · REST com auth · SOAP sem auth · SOAP com auth · cURL · Erros frequentes

Se preciso...	Vá para
Copiar um exemplo pronto de URL + chamada (REST/SOAP, com ou sem autenticação)	R. Receitas prontas
Entender camadas e onde colocar cada classe	1. Arquitetura
Montar o pom.xml e rodar o projeto	2. Setup
Configurar URL, token, perfis dev/hml/prod e .env	3. Configuração · 3.3 .env
Criar um CRUD REST com validação e paginação	4. CRUD REST
Devolver erros padronizados (400, 404, 502)	5. ProblemDetail
Chamar uma API REST externa (GET/POST/PUT/DELETE)	6. RestClient · 6.2 verbos · 6.3 erros
Criar cliente REST só com interface (sem implementar)	7. @HttpExchange
Autenticar com Basic, Bearer, API Key, OAuth2 ou mTLS	8. Autenticação · 8.2 OAuth2 · 8.3 mTLS
Configurar timeout, retry e circuit breaker	9. Resiliência
Ler todas as páginas de uma API externa	10. Paginação externa
Consumir um serviço SOAP / WSDL	11. SOAP · 11.3 manual · 11.6 Spring WS
Testar controller e client sem depender do parceiro	12. Testes
Rastrear requisições, logs e health check	13. Observabilidade
Revisar segurança antes de publicar	14. Segurança e checklist
Lembrar anotações, status HTTP e comandos cURL	15.1 Anotações · 15.2 Status · 15.3 cURL
Descobrir a causa de um 401, 415, timeout, SOAP Fault	16. Diagnóstico
Ver o que mudou nesta versão	17. Histórico

Índice completo (todas as subseções)

- 0. Como usar este manual
- R. Receitas prontas: URL e autenticação
  - R1. Anatomia da URL e onde configurar
  - R2. REST sem autenticação (GET e POST)

    - R3. REST com Basic Auth
    - R4. REST com Bearer token
    - R5. REST com API Key (header ou query)
    - R6. REST com OAuth2 (client credentials)
    - R7. SOAP sem autenticação
    - R8. SOAP com Basic Auth no HTTP
    - R9. SOAP com WS-Security (UsernameToken)
    - R10. Resumo: o que muda em cada cenário
- 1. Arquitetura mínima
- 2. Setup: dependências e comandos
- 3. Configuração — 3.1 perfis · 3.2 propriedades tipadas · 3.3 .env
- 4. CRUD REST — 4.1 DTOs · 4.2 Service · 4.3 Controller · 4.4 paginação
- 5. ProblemDetail — 5.1 exceções · 5.2 handler · 5.3 resposta
- 6. RestClient — 6.1 bean · 6.2 verbos · 6.3 erros · 6.4 log
- 7. @HttpExchange — 7.1 interface · 7.2 Boot 3 · 7.3 Boot 4
- 8. Autenticação — 8.1 Basic/API Key · 8.2 OAuth2 · 8.3 mTLS
- 9. Resiliência — 9.1 Resilience4j · 9.2 Spring 7 · 9.3 idempotência
- 10. Paginação externa — 10.1 page/size · 10.2 cursor
- 11. SOAP — 11.1 1.1 x 1.2 · 11.2 WSDL · 11.3 manual · 11.4 leitura segura · 11.5 gerar classes · 11.6 Spring WS
- 12. Testes — 12.1 MockMvc · 12.2 MockRestServiceServer · 12.3 WireMock
- 13. Observabilidade — 13.1 correlation ID · 13.2 logs · 13.3 comandos
- 14. Segurança — checklist
- 15. Colas rápidas — 15.1 anotações · 15.2 status · 15.3 cURL
- 16. Diagnóstico de problemas
- 17. Histórico de versões

0. Como usar este manual

Ache a tarefa no sumário, copie o modelo da seção e troque URL, credenciais, DTOs e regras de negócio. Cada seção traz: quando usar, o código pronto e as armadilhas comuns. Ao fim de cada seção há um link ↑ **Sumário** para voltar.

Versões-alvo: Java 17+ (21 recomendado), Spring Boot 3.2+ e 4.x, Spring Framework 6.1+ / 7.x. Diferenças entre versões aparecem marcadas como **[Boot 3]** ou **[Boot 4]**.

Regra de ouro: comece pelo contrato (OpenAPI ou WSDL), faça uma chamada fora da aplicação (cURL, Postman, SoapUI) e só depois escreva Client, Service e Controller. As receitas da seção R trazem os dois lados: o cURL para testar e o Java para colar.

↑ Sumário

R. Receitas prontas: URL e autenticação

Exemplos curtos e completos para o caso mais comum de cada cenário. Os endpoints marcados como **público** existem de verdade e podem ser chamados para testar; os demais (`api.parceiro.com.br`, `soap.parceiro.com.br`) são modelos: troque pela URL do seu parceiro.

Cenário	Endpoint de exemplo	Receita
REST GET sem auth	`https://viacep.com.br/ws/01001000/json/` (público)	R2
REST POST sem auth	`https://jsonplaceholder.typicode.com/posts` (público, simulado)	R2
REST Basic	`https://httpbin.org/basic-auth/usuario/senha` (público, teste)	R3
REST Bearer	`https://httpbin.org/bearer` (público, teste)	R4
REST API Key	`https://api.parceiro.com.br/v1/produtos`	R5
REST OAuth2	`https://auth.parceiro.com.br/oauth2/token` + API	R6
SOAP sem auth	`https://www.dataaccess.com/webservicesserver/NumberConversion.wso` (público)	R7
SOAP Basic	`https://soap.parceiro.com.br/ws/CadastroService`	R8
SOAP WS-Security	`https://soap.parceiro.com.br/ws/CadastroService`	R9

> Serviços públicos de teste podem ficar fora do ar ou mudar sem aviso. Use-os para aprender e validar o código, nunca como dependência de produção ou do pipeline (para isso, veja a seção 12).

R1. Anatomia da URL e onde configurar

```
https://api.parceiro.com.br:443/v1/clientes/123?ativo=true&pagina=0
└──┬─┘   └──────────┬─────────┘ └┬┘└┬┘└──┬───┘└┬┘ └──────────┬──────────┘
esquema          host          porta  versão  recurso  id   query string
└──────── baseUrl (config) ─────────┘└──────── uri() no código ─────────┘
```

Regra: **esquema + host + versão vão para a configuração** (mudam por ambiente); **recurso, id e query vão para o código** (fazem parte do contrato).

```
# application.yml
integracao:
  parceiro:
    url: ${PARCEIRO_URL:https://hml.api.parceiro.com.br/v1}   # padrão aponta para homologação
---
spring.config.activate.on-profile: prod
integracao.parceiro.url: ${PARCEIRO_URL}                       # em prod é obrigatório vir de fora
```

```
// baseUrl SEM barra no fim; caminho COM barra no início
RestClient http = builder.baseUrl("https://api.parceiro.com.br/v1").build();

http.get().uri("/clientes/{id}", 123)                          // path variable: escapado automaticamente
    .retrieve().body(ClienteResponse.class);

http.get().uri(b -> b.path("/clientes")                        // query string montada com builder
```

```
        .queryParam("nome", "José da Silva")                    // espaço e acento são
codificados
        .queryParam("ativo", true)
        .build())
    .retrieve().body(String.class);
```

Erro comum	Resultado	Correto
`baseUrl(".../v1/")` + `uri("/clientes")`	`//clientes` → 404 em alguns servidores	Sem barra final na base
`uri("/clientes?nome=" + nome)`	Acento/espaço/`&` quebram a URL; risco de injeção de parâmetro	`uri(b -> b.queryParam(...))` ou `{variavel}`
URL completa no código	Não muda por ambiente	`baseUrl` vindo do `application.yml`
`http://` em produção	Credencial trafega em texto puro	Sempre `https://`

R2. REST sem autenticação (GET e POST)

Teste rápido (cURL):

```
# GET público: consulta de CEP
curl -s "https://viacep.com.br/ws/01001000/json/"
# {"cep":"01001-000","logradouro":"Praça da Sé","bairro":"Sé","localidade":"São
Paulo","uf":"SP", ...}

# POST público (API de testes: simula a criação e devolve o objeto com id)
curl -s -X POST "https://jsonplaceholder.typicode.com/posts" \
  -H "Content-Type: application/json" \
  -d '{"title":"teste","body":"conteúdo","userId":1}'
```

Java:

```
integracao:
  cep:
    url: https://viacep.com.br/ws
```

```
// dto/EnderecoViaCep.java — só os campos que você usa; o resto é ignorado
@JsonIgnoreProperties(ignoreUnknown = true)
public record EnderecoViaCep(String cep, String logradouro, String bairro,
                             String localidade, String uf, Boolean erro) {}

// config/CepClientConfig.java
@Configuration
public class CepClientConfig {
  @Bean
  RestClient cepRestClient(RestClient.Builder builder,
                           @Value("${integracao.cep.url}") String url) {
    var factory = new JdkClientHttpRequestFactory(
        HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(3)).build());
    factory.setReadTimeout(Duration.ofSeconds(5));       // sem auth, mas SEMPRE com timeout
    return builder.baseUrl(url).requestFactory(factory).build();
  }
}
```

```
// client/CepClient.java
@Component
public class CepClient {
  private final RestClient http;
  public CepClient(@Qualifier("cepRestClient") RestClient http) { this.http = http; }

  public Optional<EnderecoViaCep> buscar(String cep) {
    String limpo = cep.replaceAll("\\D", "");
    if (limpo.length() != 8) throw new IllegalArgumentException("CEP deve ter 8 dígitos");
    var r = http.get().uri("/{cep}/json/", limpo)
        .retrieve()
        .body(EnderecoViaCep.class);
    return (r == null || Boolean.TRUE.equals(r.erro())) ? Optional.empty() : Optional.of(r);
  }
}
```

```
// POST sem autenticação: corpo JSON e leitura da resposta
public record PostRequest(String title, String body, int userId) {}
public record PostResponse(Long id, String title, String body, int userId) {}

PostResponse criado = RestClient.create("https://jsonplaceholder.typicode.com")  // só para
teste rápido;
    .post().uri("/posts")                                                        // em produção
use o bean
    .contentType(MediaType.APPLICATION_JSON)
    .body(new PostRequest("teste", "conteúdo", 1))
    .retrieve()
    .body(PostResponse.class);
```

> ViaCEP devolve **HTTP 200 com `{"erro": true}`** para CEP inexistente (por isso o campo `erro` no record). Muitas APIs públicas fazem isso: sempre confira o corpo, não só o status.

R3. REST com Basic Auth

```
# Teste: o httpbin responde 200 se usuario/senha baterem com a URL, 401 se não
curl -i -u usuario:senha "https://httpbin.org/basic-auth/usuario/senha"
# Equivalente com o header explícito (base64 de "usuario:senha")
curl -i -H "Authorization: Basic dXN1YXJpbzpzZW5oYQ==" "https://httpbin.org/basic-
auth/usuario/senha"
```

```
integracao:
  legado:
    url: ${LEGADO_URL:https://httpbin.org}
    usuario: ${LEGADO_USUARIO}
    senha: ${LEGADO_SENHA}
```

```
@ConfigurationProperties(prefix = "integracao.legado")
public record LegadoProperties(@NotBlank String url, @NotBlank String usuario, @NotBlank
String senha) {}

@Bean
RestClient legadoRestClient(RestClient.Builder builder, LegadoProperties p) {
  return builder.baseUrl(p.url())
```

```
      .defaultHeaders(h -> h.setBasicAuth(p.usuario(), p.senha()))   // codifica em Base64
(UTF-8)
      .build();
}

// Uso
String json = legadoRestClient.get().uri("/basic-auth/{u}/{s}", "usuario", "senha")
    .retrieve().body(String.class);
```

Basic **não criptografa** nada (Base64 é só codificação): use apenas com HTTPS.

R4. REST com Bearer token

```
# Teste: 200 com qualquer token, 401 sem o header
curl -i -H "Authorization: Bearer meu-token-de-teste" "https://httpbin.org/bearer"
```

```
@Bean
RestClient parceiroRestClient(RestClient.Builder builder, ParceiroProperties p) {
  return builder.baseUrl(p.url())
      .defaultHeader(HttpHeaders.AUTHORIZATION, "Bearer " + p.token())   // token fixo de
longa duração
      .build();
}
```

Se o token **expira** (ex.: 1 hora), não use header fixo: obtenha e renove automaticamente com OAuth2 (R6). Para token por usuário (repassar o token de quem chamou sua API), use um interceptor que leia o `Authorization` da requisição atual.

R5. REST com API Key (header ou query)

```
# No header (mais comum e mais seguro: não aparece em log de proxy/servidor)
curl -H "X-API-Key: $API_KEY" "https://api.parceiro.com.br/v1/produtos"

# Na query string (só se o fornecedor exigir)
curl "https://api.parceiro.com.br/v1/produtos?api_key=$API_KEY"
```

```
// Header: definido uma vez no bean
@Bean
RestClient produtosRestClient(RestClient.Builder builder, ProdutosProperties p) {
  return builder.baseUrl(p.url())
      .defaultHeader("X-API-Key", p.apiKey())         // nome do header conforme a
documentação do fornecedor
      .build();
}

// Query string: interceptor que acrescenta o parâmetro em toda chamada
@Bean
RestClient mapasRestClient(RestClient.Builder builder, MapasProperties p) {
  return builder.baseUrl(p.url())
      .requestInterceptor((req, body, exec) -> {
        URI comChave = UriComponentsBuilder.fromUri(req.getURI())
            .queryParam("api_key", UriUtils.encodeQueryParam(p.apiKey(),
StandardCharsets.UTF_8))
            .build(true).toUri();
```

```
        return exec.execute(new HttpRequestWrapper(req) {
          @Override public URI getURI() { return comChave; }
        }, body);
      })
      .build();
}
```

Chave na URL vaza em logs de acesso: se o fornecedor aceitar os dois, prefira o header e **não logue a URL completa** (6.4 loga só o path).

R6. REST com OAuth2 (client credentials)

Fluxo: (1) troca `client_id` + `client_secret` por um `access_token` no servidor de autorização; (2) chama a API com `Authorization: Bearer <access_token>`; (3) renova quando expira.

```
# 1. Obter o token (alguns provedores exigem as credenciais via Basic: use -u "$ID:$SECRET")
TOKEN=$(curl -s -X POST "https://auth.parceiro.com.br/oauth2/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=client_credentials&client_id=$ID&client_secret=$SECRET&scope=clientes.read" \
  | jq -r .access_token)

# 2. Chamar a API
curl -H "Authorization: Bearer $TOKEN" "https://api.parceiro.com.br/v1/clientes/123"
```

No Java, **não** implemente cache e renovação de token à mão: use a configuração pronta da seção 8.2, que obtém, guarda e renova o token sozinha. Só troque `token-uri`, `client-id`, `client-secret` e `scope`.

R7. SOAP sem autenticação

Serviço público de exemplo (converte número em texto). WSDL: `https://www.dataaccess.com/webservicesserver/NumberConversion.wso?WSDL`.

```
cat > numero.xml <<'EOF'
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:num="http://www.dataaccess.com/webservicesserver/">
  <soapenv:Body>
    <num:NumberToWords>
      <num:ubiNum>500</num:ubiNum>
    </num:NumberToWords>
  </soapenv:Body>
</soapenv:Envelope>
EOF

curl -s -X POST "https://www.dataaccess.com/webservicesserver/NumberConversion.wso" \
  -H "Content-Type: text/xml; charset=utf-8" \
  --data-binary @numero.xml
# ... <m:NumberToWordsResult>five hundred </m:NumberToWordsResult> ...
```

```
@Component
public class NumeroSoapClient {
  private static final String URL =
"https://www.dataaccess.com/webservicesserver/NumberConversion.wso";
  private static final String NS  = "http://www.dataaccess.com/webservicesserver/";
```

```
  private final RestClient http;

  public NumeroSoapClient(RestClient.Builder builder) {
    var factory = new JdkClientHttpRequestFactory(
        HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(5)).build());
    factory.setReadTimeout(Duration.ofSeconds(15));
    this.http = builder.baseUrl(URL).requestFactory(factory).build();   // em projeto real:
URL no yml
  }

  public String porExtenso(long numero) {
    String envelope = """
        <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:num="%s">
          <soapenv:Body>
            <num:NumberToWords><num:ubiNum>%d</num:ubiNum></num:NumberToWords>
          </soapenv:Body>
        </soapenv:Envelope>
        """.formatted(NS, numero);

    String resposta = http.post()
        .contentType(MediaType.parseMediaType("text/xml; charset=UTF-8"))
        .header("SOAPAction", "\"\"")                  // este serviço aceita ação vazia;
confira no WSDL do seu
        .body(envelope)
        .retrieve()
        .body(String.class);

    Document doc = SoapXml.parse(resposta);            // parser seguro da seção 11.4
    SoapXml.verificarFault(doc);
    return SoapXml.valor(doc, "NumberToWordsResult").trim();
  }
}
```

O mesmo padrão vale para qualquer SOAP sem autenticação: troque URL, namespace, operação e campos lendo o WSDL (11.2). Para valores texto, use o escape `xml()` da 11.3.

R8. SOAP com Basic Auth no HTTP

A autenticação vai no **header HTTP**, igual ao REST; o envelope não muda.

```
curl -s -X POST "https://soap.parceiro.com.br/ws/CadastroService" \
  -u "$SOAP_USUARIO:$SOAP_SENHA" \
  -H "Content-Type: text/xml; charset=utf-8" \
  -H 'SOAPAction: "ConsultarCliente"' \
  --data-binary @requisicao.xml
```

```
// Opção A — envelope manual com RestClient (seção 11.3): só acrescente o Basic no bean
this.http = builder.baseUrl(p.url())
    .requestFactory(factory)
    .defaultHeaders(h -> h.setBasicAuth(p.usuario(), p.senha()))
    .build();
```

```
// Opção B — Spring WS tipado (seção 11.6): message sender que injeta o header
public class BasicAuthMessageSender extends HttpUrlConnectionMessageSender {
```

```
  private final String authorization;

  public BasicAuthMessageSender(String usuario, String senha) {
    this.authorization = "Basic " + Base64.getEncoder()
        .encodeToString((usuario + ":" + senha).getBytes(StandardCharsets.UTF_8));
  }

  @Override
  protected void prepareConnection(HttpURLConnection connection) throws IOException {
    super.prepareConnection(connection);
    connection.setRequestProperty(HttpHeaders.AUTHORIZATION, authorization);
  }
}

// em SoapConfig.cadastroWs(...):
var sender = new BasicAuthMessageSender(p.usuario(), p.senha());
sender.setConnectionTimeout(Duration.ofSeconds(5));
sender.setReadTimeout(Duration.ofSeconds(30));
ws.setMessageSender(sender);
```

R9. SOAP com WS-Security (UsernameToken)

Aqui a credencial vai **dentro do envelope**, no `<soapenv:Header>`. O WSDL ou a documentação do parceiro indica isso com termos como *WS-Security*, *UsernameToken* ou `wsse:Security`.

```
<!-- requisicao-wsse.xml -->
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:cad="http://exemplo.com/cadastro">
  <soapenv:Header>
    <wsse:Security soapenv:mustUnderstand="1"
        xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-
1.0.xsd">
      <wsse:UsernameToken>
        <wsse:Username>usuario_integracao</wsse:Username>
        <wsse:Password Type="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-username-
token-profile-1.0#PasswordText">senha</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
  </soapenv:Header>
  <soapenv:Body>
    <cad:ConsultarCliente><cad:codigo>123</cad:codigo></cad:ConsultarCliente>
  </soapenv:Body>
</soapenv:Envelope>
```

```
curl -s -X POST "https://soap.parceiro.com.br/ws/CadastroService" \
  -H "Content-Type: text/xml; charset=utf-8" \
  -H 'SOAPAction: "ConsultarCliente"' \
  --data-binary @requisicao-wsse.xml
```

Java — opção A (envelope manual): monte o mesmo XML do exemplo acima em um text block, aplicando `xml()` (11.3) em usuário, senha e campos. Serve para `PasswordText`; para `PasswordDigest`, nonce, timestamp ou assinatura, use a opção B.

Java — opção B (Spring WS + WSS4J):

```
<dependency>
  <groupId>org.springframework.ws</groupId>
  <artifactId>spring-ws-security</artifactId>
</dependency>
```

```
@Bean
Wss4jSecurityInterceptor wsSecurity(CadastroProperties p) {
  var sec = new Wss4jSecurityInterceptor();
  sec.setSecurementActions("UsernameToken");                  // acrescente " Timestamp" se o
parceiro exigir
  sec.setSecurementUsername(p.usuario());
  sec.setSecurementPassword(p.senha());
  sec.setSecurementPasswordType(WSConstants.PW_TEXT);         // ou WSConstants.PW_DIGEST
  return sec;
}

// em SoapConfig.cadastroWs(...):
ws.setInterceptors(new ClientInterceptor[] { wsSecurity });
```

R10. Resumo: o que muda em cada cenário

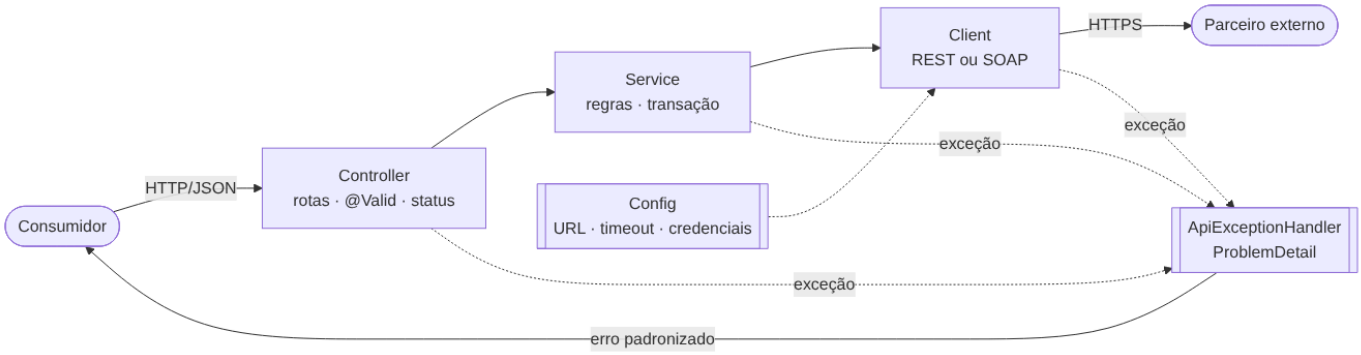
Cenário	Onde vai a credencial	Header/elemento	Java (no bean)
Sem auth	—	—	Só `baseUrl` + timeouts
Basic	Header HTTP	`Authorization: Basic base64(u:s)`	`defaultHeaders(h -> h.setBasicAuth(u, s))`
Bearer fixo	Header HTTP	`Authorization: Bearer <token>`	`defaultHeader(AUTHORIZATION, "Bearer " + t)`
API Key	Header HTTP (ou query)	`X-API-Key: <chave>`	`defaultHeader("X-API-Key", k)`
OAuth2	Header HTTP, token renovado	`Authorization: Bearer <access_token>`	`OAuth2ClientHttpRequestInterceptor` (8.2)
mTLS	Handshake TLS	— (certificado cliente)	SSL bundle (8.3)
SOAP + Basic	Header HTTP	`Authorization: Basic ...`	Igual ao REST / `BasicAuthMessageSender`
SOAP + WS-Security	Dentro do envelope	`<wsse:Security> <wsse:UsernameToken>`	`Wss4jSecurityInterceptor`

Em todos os casos: credencial vem de variável de ambiente ou cofre (3), tráfego só em HTTPS e nada disso aparece em log (14).

↑ Sumário

1. Arquitetura mínima

Cada camada tem uma responsabilidade: o Controller recebe HTTP, o Service decide, o Client fala com o parceiro. Trocar REST por SOAP ou mudar de fornecedor deve mexer só no Client.



A requisição segue da esquerda para a direita; qualquer exceção sobe até o handler global, que devolve um erro padronizado ao consumidor.

Camada	Faz	Não faz
Controller	Rotas, validação (`@Valid`), status HTTP	Regra de negócio, chamada externa
Service	Regras, transações, combinação de chamadas	Conhecer HTTP, JSON ou XML
Client	Montar requisição, autenticar, converter resposta e erros	Decidir regra de negócio
DTO	Definir o contrato (records)	Ser entidade JPA
Config	Criar beans com URL, timeout e credenciais	Conter segredo fixo no código

```
src/main/java/com/exemplo/integracao/
  IntegracaoApplication.java
  controller/ClienteController.java
  service/ClienteService.java
  client/ParceiroClient.java          # REST
  client/ParceiroApi.java             # REST declarativo (@HttpExchange)
  client/CadastroSoapClient.java      # SOAP
  dto/ClienteRequest.java
  dto/ClienteResponse.java
  dto/PaginaResponse.java
  config/ParceiroProperties.java
  config/HttpClientConfig.java
  exception/ApiExceptionHandler.java
  exception/IntegracaoException.java
src/main/resources/
  application.yml
  wsdl/cadastro.wsdl
src/test/java/...                     # espelha os pacotes acima
```

↑ Sumário

2. Setup: dependências e comandos

Gere o projeto em start.spring.io com Spring Web, Validation, Actuator e Spring Boot Test. Acrescente só o que a integração usa.

```
<!-- pom.xml (trecho) -->
<dependencies>
  <!-- REST: Controller + RestClient -->
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>

  <!-- Opcional: paginação em banco -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- Opcional: OAuth2 client credentials -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId>
  </dependency>

  <!-- Opcional: SOAP tipado (Spring WS + JAXB) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web-services</artifactId>
  </dependency>

  <!-- Opcional: circuit breaker / retry -->
  <dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
    <version>2.2.0</version>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

[Boot 4] os módulos foram divididos (ex.: `spring-boot-starter-webmvc`, `spring-boot-starter-restclient`). Se um import não resolver, gere o pom no Initializr com a versão do projeto. Confira a versão do Resilience4j compatível com seu Boot.

Tarefa	Comando (Linux/macOS)	Windows
Rodar local	`./mvnw spring-boot:run`	`mvnw.cmd spring-boot:run`
Rodar com perfil	`./mvnw spring-boot:run -Dspring-boot.run.profiles=dev`	idem com `mvnw.cmd`
Testes	`./mvnw clean test`	`mvnw.cmd clean test`

Tarefa	Comando (Linux/macOS)	Windows
Gerar JAR	`./mvnw clean package`	`mvnw.cmd clean package`
Executar JAR	`java -jar target/app.jar --spring.profiles.active=prod`	idem
Ver árvore de dependências	`./mvnw dependency:tree`	`mvnw.cmd dependency:tree`

↑ Sumário

3. Configuração: application.yml, perfis e .env

Centralize URL, timeouts e credenciais em propriedades tipadas com `@ConfigurationProperties`. Segredos sempre via variável de ambiente ou cofre, nunca no Git.

3.1 application.yml com perfis

```
# src/main/resources/application.yml (comum a todos)
server:
  port: ${PORT:8080}
spring:
  application:
    name: integracao-api
  config:
    import: optional:file:./.env[.properties]   # só ajuda em dev local

integracao:
  parceiro:
    url: ${PARCEIRO_URL}
    token: ${PARCEIRO_TOKEN}
    connect-timeout: 5s
    read-timeout: 15s

---
# Perfil dev
spring.config.activate.on-profile: dev
integracao.parceiro.url: https://hml.api.exemplo.com
logging.level.com.exemplo: DEBUG

---
# Perfil prod
spring.config.activate.on-profile: prod
logging.level.root: INFO
```

Ative com `SPRING_PROFILES_ACTIVE=prod` (variável) ou `--spring.profiles.active=prod` (argumento).

3.2 Propriedades tipadas (prefira a vários @Value)

```
// config/ParceiroProperties.java
@Validated
@ConfigurationProperties(prefix = "integracao.parceiro")
public record ParceiroProperties(
```

```
    @NotBlank String url,
    @NotBlank String token,
    Duration connectTimeout,
    Duration readTimeout) {}

// IntegracaoApplication.java
@SpringBootApplication
@ConfigurationPropertiesScan   // registra todos os @ConfigurationProperties
public class IntegracaoApplication {
  public static void main(String[] args) {
    SpringApplication.run(IntegracaoApplication.class, args);
  }
}
```

Vantagem: a aplicação falha na subida se faltar URL ou token, em vez de falhar na primeira chamada. `5s`, `500ms` e `1m` viram `Duration` automaticamente.

3.3 Arquivo .env

O Spring Boot **não** lê `.env` sozinho. Escolha uma opção:

Opção	Como	Observação
A. Importar como properties	`spring.config.import=optional:file:./.env[.properties]`	Caminho relativo ao diretório de execução; aceita só `CHAVE=valor` simples
B. Bash	`set -a; source ./.env; set +a` e depois `./mvnw spring-boot:run`	Executa o arquivo como shell: só arquivos confiáveis
B. PowerShell	`Get-Content .env | % { $k,$v = $_ -split '=',2; [Environment]::SetEnvironmentVariable($k,$v) }`	Vale para a sessão atual
B. Docker	`docker run --env-file .env -p 8080:8080 app:latest`	Recomendado em contêineres
B. IntelliJ	Run/Debug Configurations → Environment variables	Ou plugin EnvFile

```
# .env (NÃO versionar)
PARCEIRO_URL=https://api.exemplo.com
PARCEIRO_TOKEN=coloque_token_local_aqui
DB_URL=jdbc:postgresql://localhost:5432/app
```

```
# .gitignore
.env
*.jks
*.p12
```

Mantenha um `.env.example` versionado com as mesmas chaves e valores vazios.

Nome da variável: `PARCEIRO_URL` só chega em `integracao.parceiro.url` porque o yml tem o placeholder `${PARCEIRO_URL}`. Sem placeholder, o Spring também aceita o nome "relaxado" da própria propriedade: `INTEGRACAO_PARCEIRO_URL` (maiúsculas, `.` e `-` viram `_`).

Propriedade não resolve? Confira o nome, o diretório de execução, o perfil ativo e `/actuator/env` (se exposto apenas em dev).

↑ Sumário

4. Expor uma API REST: CRUD completo

Modelo de CRUD com DTOs separados para entrada e saída, validação, status corretos e paginação estável. Nunca exponha a entidade JPA direto no contrato.

4.1 DTOs

```
// dto/ClienteRequest.java — entrada (POST/PUT)
public record ClienteRequest(
    @NotBlank @Size(max = 120) String nome,
    @NotBlank @Email String email,
    @Pattern(regexp = "\\d{11}") String cpf) {}

// dto/ClienteResponse.java — saída
public record ClienteResponse(Long id, String nome, String email, Instant criadoEm) {
  public static ClienteResponse de(Cliente c) {
    return new ClienteResponse(c.getId(), c.getNome(), c.getEmail(), c.getCriadoEm());
  }
}

// dto/PaginaResponse.java — contrato de paginação estável
public record PaginaResponse<T>(List<T> itens, int pagina, int tamanho,
                                long totalItens, int totalPaginas) {
  public static <T> PaginaResponse<T> de(Page<T> p) {
    return new PaginaResponse<>(p.getContent(), p.getNumber(), p.getSize(),
                                p.getTotalElements(), p.getTotalPages());
  }
}
```

4.2 Service

```
@Service
@Transactional(readOnly = true)
public class ClienteService {
  private final ClienteRepository repo;
  public ClienteService(ClienteRepository repo) { this.repo = repo; }

  public PaginaResponse<ClienteResponse> listar(Pageable pageable) {
    return PaginaResponse.de(repo.findAll(pageable).map(ClienteResponse::de));
  }

  public ClienteResponse buscar(Long id) {
    return ClienteResponse.de(obter(id));
  }

  @Transactional
  public ClienteResponse criar(ClienteRequest req) {
    if (repo.existsByEmail(req.email())) throw new ConflitoException("E-mail já cadastrado");
    var c = new Cliente(req.nome(), req.email(), req.cpf());
```

```
    return ClienteResponse.de(repo.save(c));
  }

  @Transactional
  public ClienteResponse atualizar(Long id, ClienteRequest req) {
    var c = obter(id);
    c.atualizar(req.nome(), req.email(), req.cpf());   // dirty checking salva
    return ClienteResponse.de(c);
  }

  @Transactional
  public void excluir(Long id) {
    repo.delete(obter(id));
  }

  private Cliente obter(Long id) {
    return repo.findById(id)
        .orElseThrow(() -> new NaoEncontradoException("Cliente " + id + " não encontrado"));
  }
}

// repository/ClienteRepository.java
public interface ClienteRepository extends JpaRepository<Cliente, Long> {
  boolean existsByEmail(String email);
}
```

4.3 Controller

```
@RestController
@RequestMapping("/api/v1/clientes")
public class ClienteController {
  private final ClienteService service;
  public ClienteController(ClienteService service) { this.service = service; }

  // GET /api/v1/clientes?page=0&size=20&sort=nome,asc
  @GetMapping
  public PaginaResponse<ClienteResponse> listar(
      @PageableDefault(size = 20, sort = "nome") Pageable pageable) {
    return service.listar(pageable);
  }

  @GetMapping("/{id}")
  public ClienteResponse buscar(@PathVariable Long id) {
    return service.buscar(id);                      // 200 ou 404 (via handler)
  }

  @PostMapping
  public ResponseEntity<ClienteResponse> criar(@Valid @RequestBody ClienteRequest req,
                                               UriComponentsBuilder uri) {
    var criado = service.criar(req);
    var location = uri.path("/api/v1/clientes/{id}").buildAndExpand(criado.id()).toUri();
    return ResponseEntity.created(location).body(criado);   // 201 + Location
  }

  @PutMapping("/{id}")
  public ClienteResponse atualizar(@PathVariable Long id, @Valid @RequestBody ClienteRequest
req) {
```

```
    return service.atualizar(id, req);
  }

  @DeleteMapping("/{id}")
  @ResponseStatus(HttpStatus.NO_CONTENT)            // 204
  public void excluir(@PathVariable Long id) {
    service.excluir(id);
  }
}
```

4.4 Limitar paginação e ordenação

```
spring:
  data:
    web:
      pageable:
        max-page-size: 100      # impede ?size=1000000
        default-page-size: 20
```

Se você devolver `Page<T>` direto (em vez de `PaginaResponse`), use `@EnableSpringDataWebSupport(pageSerializationMode = VIA_DTO)` para um JSON estável. Ordenação por campo arbitrário pode expor colunas internas: valide os campos de `sort` aceitos.

Operação	Verbo e rota	Sucesso	Erros típicos
Listar	`GET /clientes?page=0&size=20`	200	400
Buscar	`GET /clientes/{id}`	200	404
Criar	`POST /clientes`	201 + `Location`	400, 409
Substituir	`PUT /clientes/{id}`	200 ou 204	400, 404
Atualizar parte	`PATCH /clientes/{id}`	200	400, 404
Excluir	`DELETE /clientes/{id}`	204	404

↑ Sumário

5. Erros da API própria: ProblemDetail

Use `ProblemDetail` (RFC 9457, nativo no Spring 6+) para que todo erro tenha o mesmo formato JSON. Um único `@RestControllerAdvice` traduz exceções em status.

5.1 Exceções de negócio

```
// exception/NaoEncontradoException.java
public class NaoEncontradoException extends RuntimeException {
  public NaoEncontradoException(String msg) { super(msg); }
}
// exception/ConflitoException.java
public class ConflitoException extends RuntimeException {
  public ConflitoException(String msg) { super(msg); }
}
// exception/IntegracaoException.java — falha no sistema externo
public class IntegracaoException extends RuntimeException {
```

```
  private final int statusParceiro;      // 0 = sem resposta (timeout/conexão)
  public IntegracaoException(String msg, int statusParceiro, Throwable causa) {
    super(msg, causa);
    this.statusParceiro = statusParceiro;
  }
  public int statusParceiro() { return statusParceiro; }
}
```

5.2 Handler global

```
@RestControllerAdvice
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {
  private static final Logger log = LoggerFactory.getLogger(ApiExceptionHandler.class);

  @ExceptionHandler(NaoEncontradoException.class)
  ProblemDetail naoEncontrado(NaoEncontradoException ex) {
    return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
  }

  @ExceptionHandler(ConflitoException.class)
  ProblemDetail conflito(ConflitoException ex) {
    return ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, ex.getMessage());
  }

  @ExceptionHandler(IntegracaoException.class)
  ProblemDetail integracao(IntegracaoException ex) {
    log.error("Falha na integração: status={} msg={}", ex.statusParceiro(), ex.getMessage(),
ex);
    var status = ex.statusParceiro() == 0 ? HttpStatus.GATEWAY_TIMEOUT :
HttpStatus.BAD_GATEWAY;
    var pd = ProblemDetail.forStatusAndDetail(status, "Falha na integração externa");
    pd.setTitle("Serviço externo indisponível");
    return pd;                              // sem stack trace, token ou payload
  }

  // 400 de @Valid: lista campo a campo
  @Override
  protected ResponseEntity<Object> handleMethodArgumentNotValid(
      MethodArgumentNotValidException ex, HttpHeaders headers,
      HttpStatusCode status, WebRequest request) {
    var pd = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, "Dados inválidos");
    pd.setProperty("campos", ex.getBindingResult().getFieldErrors().stream()
        .map(e -> Map.of("campo", e.getField(), "erro", e.getDefaultMessage()))
        .toList());
    return ResponseEntity.badRequest().body(pd);
  }

  @ExceptionHandler(Exception.class)
  ProblemDetail inesperado(Exception ex) {
    log.error("Erro não tratado", ex);
    return ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR, "Erro interno");
  }
}
```

Se não for estender `ResponseEntityExceptionHandler`, ative `spring.mvc.problemdetails.enabled=true` para que os erros do próprio Spring (405, 415 etc.) também saiam neste formato.

5.3 Resposta gerada

```
{
  "type": "about:blank",
  "title": "Bad Request",
  "status": 400,
  "detail": "Dados inválidos",
  "instance": "/api/v1/clientes",
  "campos": [ { "campo": "email", "erro": "deve ser um endereço de e-mail bem formado" } ]
}
```

Exceção	Status devolvido
`MethodArgumentNotValidException` (@Valid)	400
`NaoEncontradoException`	404
`ConflitoException`	409
`IntegracaoException` com resposta do parceiro	502
`IntegracaoException` por timeout/conexão	504
Qualquer outra	500

↑ Sumário

6. Consumir REST com RestClient

`RestClient` é o cliente síncrono recomendado (Spring 6.1+). Configure um bean por parceiro, com URL, timeouts, autenticação e tratamento de erro definidos uma vez. `RestTemplate` segue funcionando, mas não use em código novo; `WebClient` só em aplicações reativas.

6.1 Bean configurado (um por parceiro)

```
// config/HttpClientConfig.java
@Configuration
public class HttpClientConfig {

  @Bean
  RestClient parceiroRestClient(RestClient.Builder builder, ParceiroProperties props) {
    var http = HttpClient.newBuilder()                 // java.net.http (JDK 11+)
        .connectTimeout(props.connectTimeout())
        .build();
    var factory = new JdkClientHttpRequestFactory(http);
    factory.setReadTimeout(props.readTimeout());

    return builder                                       // use o Builder injetado: herda
        .baseUrl(props.url())                            // Jackson, métricas e tracing
        .requestFactory(factory)
        .defaultHeader(HttpHeaders.AUTHORIZATION, "Bearer " + props.token())
        .defaultHeader(HttpHeaders.ACCEPT, MediaType.APPLICATION_JSON_VALUE)
        .defaultStatusHandler(HttpStatusCode::isError, (req, res) -> {
          throw new IntegracaoException(
              "Parceiro respondeu " + res.getStatusCode().value(),
              res.getStatusCode().value(), null);
        })
```

```
        .build();
  }
}
```

Dois ou mais parceiros? Crie um bean por parceiro com nomes distintos e injete com `@Qualifier("parceiroRestClient")`. Depender só do nome do parâmetro é frágil.

6.2 Client com GET, POST, PUT, PATCH e DELETE

```
// client/ParceiroClient.java
@Component
public class ParceiroClient {
  private final RestClient http;

  public ParceiroClient(@Qualifier("parceiroRestClient") RestClient http) {
    this.http = http;
  }

  // GET por id
  public ClienteResponse buscar(Long id) {
    return http.get().uri("/clientes/{id}", id)
        .retrieve()
        .body(ClienteResponse.class);
  }

  // GET que trata 404 como "não existe" em vez de erro
  public Optional<ClienteResponse> buscarOpcional(Long id) {
    return http.get().uri("/clientes/{id}", id)
        .exchange((req, res) -> res.getStatusCode().value() == 404
            ? Optional.empty()
            : Optional.ofNullable(res.bodyTo(ClienteResponse.class)));
  }

  // GET com query params e lista genérica
  public List<ClienteResponse> listar(boolean ativo, int pagina) {
    return http.get()
        .uri(b -> b.path("/clientes")
            .queryParam("ativo", ativo)
            .queryParam("pagina", pagina)
            .build())
        .retrieve()
        .body(new ParameterizedTypeReference<List<ClienteResponse>>() {});
  }

  // POST com corpo JSON + header extra + leitura de status/headers
  public ClienteResponse criar(ClienteRequest dados, String correlationId) {
    ResponseEntity<ClienteResponse> resp = http.post().uri("/clientes")
        .contentType(MediaType.APPLICATION_JSON)
        .header("X-Correlation-ID", correlationId)
        .body(dados)                                    // serializado pelo Jackson
        .retrieve()
        .toEntity(ClienteResponse.class);
    URI location = resp.getHeaders().getLocation();     // se precisar
    return resp.getBody();
  }

  // PUT sem corpo de resposta
```

```
  public void substituir(Long id, ClienteRequest dados) {
    http.put().uri("/clientes/{id}", id)
        .contentType(MediaType.APPLICATION_JSON)
        .body(dados)
        .retrieve()
        .toBodilessEntity();
  }

  // PATCH parcial
  public void atualizarEmail(Long id, String email) {
    http.patch().uri("/clientes/{id}", id)
        .contentType(MediaType.APPLICATION_JSON)
        .body(Map.of("email", email))
        .retrieve()
        .toBodilessEntity();
  }

  // DELETE
  public void excluir(Long id) {
    http.delete().uri("/clientes/{id}", id)
        .retrieve()
        .toBodilessEntity();
  }
}
```

6.3 Tratar erros por chamada

```
// Erro específico só nesta chamada (sobrepõe o defaultStatusHandler)
http.post().uri("/pedidos").body(pedido)
    .retrieve()
    .onStatus(s -> s.value() == 422, (req, res) -> {
      throw new ConflitoException(new String(res.getBody().readAllBytes(),
StandardCharsets.UTF_8));
    })
    .toBodilessEntity();

// Sem defaultStatusHandler: capture as exceções padrão
try {
  return http.get().uri("/clientes/{id}", id).retrieve().body(ClienteResponse.class);
} catch (HttpClientErrorException.NotFound ex) {          // 404
  throw new NaoEncontradoException("Cliente não existe no parceiro");
} catch (RestClientResponseException ex) {                 // demais 4xx/5xx
  throw new IntegracaoException("Parceiro retornou erro", ex.getStatusCode().value(), ex);
} catch (ResourceAccessException ex) {                     // timeout, DNS, conexão recusada
  throw new IntegracaoException("Falha de conexão/timeout", 0, ex);
}
```

6.4 Log seguro de requisições (interceptor)

```
class LogInterceptor implements ClientHttpRequestInterceptor {
  private static final Logger log = LoggerFactory.getLogger(LogInterceptor.class);
  @Override
  public ClientHttpResponse intercept(HttpRequest req, byte[] body,
                                      ClientHttpRequestExecution exec) throws IOException {
    long ini = System.nanoTime();
    var res = exec.execute(req, body);
```

```
    log.info("HTTP {} {} -> {} em {} ms", req.getMethod(), req.getURI().getPath(),
        res.getStatusCode().value(), (System.nanoTime() - ini) / 1_000_000);
    return res;                       // nunca logue Authorization nem o corpo com dados
pessoais
  }
}
// no builder: .requestInterceptor(new LogInterceptor())
```

Preciso de...	Use
Só o corpo	`.retrieve().body(Tipo.class)`
Corpo + status + headers	`.retrieve().toEntity(Tipo.class)`
Sem corpo (PUT/DELETE)	`.retrieve().toBodilessEntity()`
Lista ou genérico	`.body(new ParameterizedTypeReference<List<T>>() {})`
JSON cru para inspecionar	`.body(String.class)` ou `.body(JsonNode.class)`
Controle total da resposta	`.exchange((req, res) -> ...)`

↑ Sumário

7. Cliente declarativo com @HttpExchange

Descreva a API externa como uma interface Java; o Spring gera a implementação sobre o `RestClient` da seção 6. Bom para APIs com muitos endpoints simples (é o equivalente nativo do Feign).

7.1 Interface

```
// client/ParceiroApi.java
@HttpExchange(url = "/clientes", accept = MediaType.APPLICATION_JSON_VALUE)
public interface ParceiroApi {

  @GetExchange("/{id}")
  ClienteResponse buscar(@PathVariable Long id);

  @GetExchange
  List<ClienteResponse> listar(@RequestParam boolean ativo,
                               @RequestParam(defaultValue = "0") int pagina);

  @PostExchange(contentType = MediaType.APPLICATION_JSON_VALUE)
  ResponseEntity<ClienteResponse> criar(@RequestBody ClienteRequest dados,
                                        @RequestHeader("X-Correlation-ID") String cid);

  @PutExchange("/{id}")
  void substituir(@PathVariable Long id, @RequestBody ClienteRequest dados);

  @DeleteExchange("/{id}")
  void excluir(@PathVariable Long id);
}
```

7.2 Registro [Boot 3]

```
@Configuration
public class ParceiroApiConfig {
  @Bean
  ParceiroApi parceiroApi(@Qualifier("parceiroRestClient") RestClient restClient) {
    var factory = HttpServiceProxyFactory
        .builderFor(RestClientAdapter.create(restClient))
        .build();
    return factory.createClient(ParceiroApi.class);
  }
}
```

7.3 Registro [Boot 4]

O Spring Framework 7 permite registrar grupos de interfaces com `@ImportHttpServices` e o Boot 4 configura URL, timeout e headers por propriedade, sem `@Bean`. O registro manual de 7.2 continua válido.

```
@Configuration
@ImportHttpServices(group = "parceiro", types = ParceiroApi.class)   // ou basePackages =
"...client.parceiro"
public class ParceiroApiConfig {}
```

```
spring:
  http:
    serviceclient:
      parceiro:                                    # mesmo nome do group
        base-url: ${PARCEIRO_URL}
        connect-timeout: 5s
        read-timeout: 15s
        default-header:
          Authorization: "Bearer ${PARCEIRO_TOKEN}"
```

O prefixo dessas propriedades mudou entre os milestones do Boot 4 (`spring.http.client.service.group.*` → `spring.http.serviceclient.*`): se não surtir efeito, confira em `/actuator/configprops` ou na referência da sua versão. Para lógica extra (interceptor, `defaultStatusHandler`), declare um bean `RestClientHttpServiceGroupConfigurer`.

Uso: injete `ParceiroApi` no Service como qualquer bean. Timeouts, autenticação e `defaultStatusHandler` vêm do `RestClient` subjacente.

Use RestClient direto quando	Use @HttpExchange quando
Poucas chamadas ou lógica de resposta específica (`exchange`, `onStatus`)	Muitos endpoints com padrão repetitivo
SOAP ou XML montado manualmente	Contrato REST estável e bem definido
Precisa variar headers por chamada com lógica	Quer mockar a interface facilmente nos testes

↑ Sumário

8. Autenticação em chamadas de saída

Defina a autenticação no bean do `RestClient`, não em cada chamada. Credenciais sempre por variável de ambiente ou cofre, e somente sobre HTTPS. Exemplos com URL real e cURL para testar estão nas receitas R3 a R6.

Tipo	Header enviado	Onde configurar
Basic	`Authorization: Basic base64(usuario:senha)`	`defaultHeaders(h -> h.setBasicAuth(u, s))`
Bearer fixo	`Authorization: Bearer <token>`	`defaultHeader(AUTHORIZATION, "Bearer " + token)`
API Key	`X-API-Key: <chave>` (nome conforme o fornecedor)	`defaultHeader("X-API-Key", chave)`
OAuth2 client credentials	`Authorization: Bearer <access_token>` renovado sozinho	`OAuth2ClientHttpRequestInterceptor` (8.2)
mTLS (certificado cliente)	Nenhum header; autentica no TLS	SSL bundle (8.3)

8.1 Basic e API Key

```
@Bean
RestClient legadoRestClient(RestClient.Builder builder, LegadoProperties p) {
  return builder.baseUrl(p.url())
      .defaultHeaders(h -> h.setBasicAuth(p.usuario(), p.senha()))
      .build();
}

@Bean
RestClient cepRestClient(RestClient.Builder builder, CepProperties p) {
  return builder.baseUrl(p.url())
      .defaultHeader("X-API-Key", p.apiKey())
      .build();
}
```

8.2 OAuth2 client credentials (token renovado automaticamente)

Requer `spring-boot-starter-oauth2-client` e Spring Security 6.4+.

```
spring:
  security:
    oauth2:
      client:
        registration:
          parceiro:
            provider: parceiro
            client-id: ${PARCEIRO_CLIENT_ID}
            client-secret: ${PARCEIRO_CLIENT_SECRET}
            authorization-grant-type: client_credentials
            scope: clientes.read,clientes.write
        provider:
```

```
          parceiro:
            token-uri: https://auth.exemplo.com/oauth2/token
```

```
@Configuration
public class OAuth2ClientConfig {

  // Funciona também fora de requisições HTTP (jobs, @Scheduled, filas)
  @Bean
  OAuth2AuthorizedClientManager authorizedClientManager(
      ClientRegistrationRepository registrations,
      OAuth2AuthorizedClientService clientService) {
    var manager = new AuthorizedClientServiceOAuth2AuthorizedClientManager(registrations,
clientService);
    manager.setAuthorizedClientProvider(
        OAuth2AuthorizedClientProviderBuilder.builder().clientCredentials().build());
    return manager;
  }

  @Bean
  RestClient parceiroOAuthRestClient(RestClient.Builder builder,
                                     OAuth2AuthorizedClientManager manager,
                                     ParceiroProperties props) {
    var oauth = new OAuth2ClientHttpRequestInterceptor(manager);
    oauth.setClientRegistrationIdResolver(req -> "parceiro");   // id do registration
    return builder.baseUrl(props.url())
        .requestInterceptor(oauth)
        .build();
  }
}
```

Atenção: adicionar o starter de OAuth2/Security protege **todos** os seus endpoints por padrão. Declare um `SecurityFilterChain` com as regras de acesso da sua API.

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
  return http
      .csrf(csrf -> csrf.disable())                        // API stateless
      .authorizeHttpRequests(a -> a
          .requestMatchers("/actuator/health").permitAll()
          .anyRequest().authenticated())
      .oauth2ResourceServer(rs -> rs.jwt(Customizer.withDefaults()))  // se a API recebe JWT
      .build();
}
```

A linha `oauth2ResourceServer` exige `spring-boot-starter-oauth2-resource-server` e `spring.security.oauth2.resourceserver.jwt.issuer-uri`. Se sua API ainda não valida JWT de entrada, remova essa linha e defina a regra de acesso adequada (nunca `permitAll()` em tudo sem decisão explícita).

8.3 mTLS com SSL bundle

```
spring:
  ssl:
    bundle:
      jks:
```

```
        parceiro:
          keystore:
            location: file:/etc/certs/cliente.p12
            password: ${KEYSTORE_PASSWORD}
            type: PKCS12
          truststore:
            location: file:/etc/certs/truststore.p12
            password: ${TRUSTSTORE_PASSWORD}
```

```
@Bean
RestClient mtlsRestClient(RestClient.Builder builder, RestClientSsl ssl, ParceiroProperties p)
{
  return builder.baseUrl(p.url())
      .apply(ssl.fromBundle("parceiro"))
      .build();
}
```

Nunca desligue a validação de certificado ("trust all") para contornar `SSLHandshakeException`: importe a cadeia correta no truststore.

↑ Sumário

9. Resiliência: timeout, retry, idempotência e circuit breaker

Toda chamada externa precisa de timeout. Retry só para falhas transitórias e operações seguras de repetir. Circuit breaker evita derrubar sua API junto com o parceiro.

Conceito	Regra prática	Valor inicial sugerido
Connect timeout	Tempo para abrir a conexão	2–5 s
Read timeout	Espera pela resposta	5–30 s, conforme o SLA do parceiro
Retry	Só em timeout, 502, 503, 504, 429	3 tentativas no total
Backoff	Intervalo crescente + jitter entre tentativas	500 ms → 1 s → 2 s
Idempotência	`Idempotency-Key` em POST que cria ou movimenta	UUID por operação de negócio
Circuit breaker	Abre após taxa de falha alta; testa depois de um tempo	50% de falha em 20 chamadas; 30 s aberto

Nunca repita POST financeiro, de cadastro ou movimentação sem idempotência. Timeout não prova que a operação falhou no servidor remoto. Não faça retry de 400, 401, 403, 404 ou 422.

9.1 Resilience4j (Boot 3 e 4)

Requer `resilience4j-spring-boot3` e `spring-boot-starter-aop` (**[Boot 4]** `spring-boot-starter-aspectj`).

```
resilience4j:
  retry:
    instances:
      parceiro:
```

```
        max-attempts: 3
        wait-duration: 500ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        enable-randomized-wait: true          # jitter
        retry-exceptions:
          - org.springframework.web.client.ResourceAccessException
          - org.springframework.web.client.HttpServerErrorException
        ignore-exceptions:
          - org.springframework.web.client.HttpClientErrorException
  circuitbreaker:
    instances:
      parceiro:
        sliding-window-size: 20
        minimum-number-of-calls: 10
        failure-rate-threshold: 50
        wait-duration-in-open-state: 30s
        permitted-number-of-calls-in-half-open-state: 3
```

```
@Service
public class ConsultaParceiroService {
  private final ParceiroClient client;
  public ConsultaParceiroService(ParceiroClient client) { this.client = client; }

  // Ordem de execução: Retry (externo) envolve CircuitBreaker (interno).
  // O fallback fica no Retry: se ficasse no CircuitBreaker, a exceção seria trocada
  // ANTES de o Retry vê-la e nenhuma nova tentativa aconteceria.
  @CircuitBreaker(name = "parceiro")
  @Retry(name = "parceiro", fallbackMethod = "fallback")
  public ClienteResponse buscar(Long id) {
    return client.buscar(id);          // só GET: seguro repetir
  }

  // Mesma assinatura + a exceção no fim
  private ClienteResponse fallback(Long id, Throwable ex) {
    throw new IntegracaoException("Parceiro indisponível", 0, ex);
  }
}
```

Com o circuito aberto, o Resilience4j lança `CallNotPermittedException`; ela não está em `retry-exceptions`, então cai direto no fallback sem novas tentativas (comportamento desejado).

As anotações funcionam por proxy: chamar `buscar` de dentro da mesma classe **não** aplica retry nem circuit breaker. O client configurado com `defaultStatusHandler` (seção 6.1) lança `IntegracaoException`; nesse caso inclua essa exceção em `retry-exceptions` ou deixe o handler padrão do Spring.

9.2 Retry nativo [Boot 4 / Spring Framework 7]

```
@Configuration
@EnableResilientMethods
public class ResilienciaConfig {}

@Service
public class ConsultaParceiroService {
  private final ParceiroClient client;
```

```
  public ConsultaParceiroService(ParceiroClient client) { this.client = client; }

  // org.springframework.resilience.annotation.Retryable (não confundir com o do spring-retry)
  // maxRetries = novas tentativas APÓS a primeira: 2 → até 3 chamadas no total
  @Retryable(maxRetries = 2, delay = 500, multiplier = 2, jitter = 100, maxDelay = 5000,
             includes = ResourceAccessException.class)
  public ClienteResponse buscar(Long id) { return client.buscar(id); }
}
```

Não precisa de dependência extra nem de AOP starter. No antigo `spring-retry` o atributo era `maxAttempts` e contava a primeira chamada; no Spring 7 é `maxRetries` e não conta.

9.3 POST idempotente

```
public PedidoResponse criarPedido(PedidoRequest pedido) {
  // A chave identifica a operação de negócio: gere uma vez e persista com o pedido,
  // para reutilizar a MESMA chave em qualquer nova tentativa.
  String chave = pedido.idempotencyKey();
  return http.post().uri("/pedidos")
      .header("Idempotency-Key", chave)
      .contentType(MediaType.APPLICATION_JSON)
      .body(pedido)
      .retrieve()
      .body(PedidoResponse.class);
}
```

O parceiro precisa suportar o header. Se não suportar, após um timeout consulte o status (GET por código externo) antes de reenviar.

↑ Sumário

10. Paginação externa: ler todas as páginas com segurança

Confirme no contrato qual estilo o parceiro usa e adapte o DTO. O loop precisa de limite máximo de páginas e tratamento de resposta vazia para nunca rodar para sempre.

Estilo	Parâmetros típicos	Fim da leitura
page/size	`?page=0&size=100`	`page + 1 >= totalPages` ou lista vazia
offset/limit	`?offset=200&limit=100`	itens recebidos `< limit`
cursor/token	`?cursor=abc123`	`nextCursor` nulo
links	header `Link: <...>; rel="next"` ou campo `next`	sem link `next`

10.1 page/size

```
public record PaginaClientes(List<ClienteDto> content, int number,
                             int totalPages, long totalElements) {}

public PaginaClientes consultar(int page, int size) {
  return http.get()
      .uri(b -> b.path("/clientes").queryParam("page", page).queryParam("size", size).build())
      .retrieve()
```

```
      .body(PaginaClientes.class);
}

public void processarTodos(Consumer<ClienteDto> processar) {
  final int tamanho = 100, maxPaginas = 1_000;       // trava contra loop infinito
  for (int pagina = 0; pagina < maxPaginas; pagina++) {
    var resp = consultar(pagina, tamanho);
    if (resp == null || resp.content() == null || resp.content().isEmpty()) return;
    resp.content().forEach(processar);
    if (pagina + 1 >= resp.totalPages()) return;
  }
  throw new IllegalStateException("Limite de " + maxPaginas + " páginas atingido");
}
```

10.2 cursor

```
public record PaginaCursor(List<ClienteDto> itens, String nextCursor) {}

public void processarTodosCursor(Consumer<ClienteDto> processar) {
  String cursor = null;
  for (int i = 0; i < 1_000; i++) {
    final String atual = cursor;
    var resp = http.get()
        .uri(b -> {
          b.path("/clientes").queryParam("limit", 100);
          if (atual != null) b.queryParam("cursor", atual);
          return b.build();
        })
        .retrieve().body(PaginaCursor.class);
    if (resp == null) return;
    resp.itens().forEach(processar);
    if (resp.nextCursor() == null || resp.nextCursor().equals(atual)) return;  // fim ou
cursor repetido
    cursor = resp.nextCursor();
  }
  throw new IllegalStateException("Limite de páginas atingido");
}
```

Em cargas grandes, processe e descarte cada página (não acumule tudo em memória), respeite `429` com `Retry-After` e registre a última página/cursor processado para retomar após falha. Paginação por `page` pode pular ou repetir itens se os dados mudarem durante a leitura; prefira cursor quando disponível.

↑ Sumário

11. Consumir SOAP

Para uma ou duas operações simples, monte o envelope e envie com `RestClient` (11.3). Para contratos extensos, gere classes a partir do WSDL e use Spring WS com cliente tipado (11.6). Sempre leia o WSDL antes: ele define operações, namespaces, SOAPAction e endereço. Exemplo executável com serviço público em R7; autenticação SOAP em R8 e R9.

11.1 SOAP 1.1 x 1.2

Item	SOAP 1.1	SOAP 1.2
Namespace do envelope	`http://schemas.xmlsoap.org/soap/envelope/`	`http://www.w3.org/2003/05/soap-envelope`
Content-Type	`text/xml; charset=UTF-8`	`application/soap+xml; charset=UTF-8; action="..."`
Ação	Header `SOAPAction: "..."` (com aspas)	Parâmetro `action` no Content-Type
Erro	`<soap:Fault>` com `faultcode` e `faultstring`	`<soap:Fault>` com `Code/Value` e `Reason/Text`
No WSDL	`soap:binding` / `soap:address`	`soap12:binding` / `soap12:address`

11.2 Onde achar cada dado no WSDL

Preciso de	Procure no WSDL
URL do serviço	`<service>` → `<soap:address location="..."/>`
Nome da operação	`<portType>` → `<operation name="...">`
SOAPAction	`<binding>` → `<soap:operation soapAction="..."/>`
Namespace dos elementos	`targetNamespace` do `<xsd:schema>` em `<types>`
Campos da requisição	`<xsd:element>` referenciado pela `<message>` de entrada

11.3 Chamada manual com RestClient

```
@Component
public class CadastroSoapClient {
  private static final String NS = "http://exemplo.com/cadastro";   // do WSDL
  private final RestClient http;

  public CadastroSoapClient(RestClient.Builder builder, CadastroProperties p) {
    var factory = new JdkClientHttpRequestFactory(
        HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(5)).build());
    factory.setReadTimeout(Duration.ofSeconds(30));
    this.http = builder.baseUrl(p.url()).requestFactory(factory).build();
  }

  public String consultarCliente(String codigo) {
    String envelope = """
        <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                          xmlns:cad="%s">
          <soapenv:Header/>
          <soapenv:Body>
            <cad:ConsultarCliente>
              <cad:codigo>%s</cad:codigo>
            </cad:ConsultarCliente>
          </soapenv:Body>
        </soapenv:Envelope>
        """.formatted(NS, xml(codigo));               // SEMPRE escape valores dinâmicos

    return http.post()
        .contentType(MediaType.parseMediaType("text/xml; charset=UTF-8"))
```

```
        .header("SOAPAction", "\"ConsultarCliente\"")    // aspas fazem parte do valor
        .body(envelope)
        .exchange((req, res) -> {
          // Muitos servidores devolvem HTTP 500 com SOAP Fault no corpo: leia antes de
decidir
          String corpo = new String(res.getBody().readAllBytes(), StandardCharsets.UTF_8);
          if (res.getStatusCode().is2xxSuccessful() || corpo.contains("Fault")) return corpo;
          throw new IntegracaoException("SOAP HTTP " + res.getStatusCode().value(),
                                        res.getStatusCode().value(), null);
        });
  }

  // Escape mínimo para conteúdo de texto XML
  static String xml(String v) {
    if (v == null) return "";
    return v.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
            .replace("\"", "&quot;").replace("'", "&apos;");
  }
}
```

Para SOAP 1.2, troque o namespace do envelope e use `MediaType.parseMediaType("application/soap+xml; charset=UTF-8; action=\"ConsultarCliente\"")`, sem o header `SOAPAction`.

11.4 Ler a resposta com segurança (XXE desligado)

```
public final class SoapXml {

  public static Document parse(String xml) {
    try {
      var f = DocumentBuilderFactory.newInstance();
      f.setNamespaceAware(true);
      f.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);   //
bloqueia XXE
      f.setFeature("http://xml.org/sax/features/external-general-entities", false);
      f.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
      f.setXIncludeAware(false);
      f.setExpandEntityReferences(false);
      return f.newDocumentBuilder().parse(new InputSource(new StringReader(xml)));
    } catch (Exception e) {
      throw new IntegracaoException("XML inválido do parceiro", 0, e);
    }
  }

  // XPath por local-name(): ignora prefixos, que variam entre servidores
  public static String valor(Document doc, String elemento) {
    try {
      var xp = XPathFactory.newInstance().newXPath();
      return xp.evaluate("//*[local-name()='" + elemento + "']/text()", doc);
    } catch (XPathExpressionException e) {
      throw new IllegalArgumentException(e);
    }
  }

  // Lança exceção se houver Fault (1.1 ou 1.2)
  public static void verificarFault(Document doc) {
    if (doc.getElementsByTagNameNS("*", "Fault").getLength() == 0) return;
```

```
    String codigo = valor(doc, "faultcode");                    // 1.1
    String msg = valor(doc, "faultstring");
    if (msg.isBlank()) { codigo = valor(doc, "Value"); msg = valor(doc, "Text"); }  // 1.2
    throw new IntegracaoException("SOAP Fault " + codigo + ": " + msg, 500, null);
  }
}

// Uso
Document doc = SoapXml.parse(cadastroSoapClient.consultarCliente("123"));
SoapXml.verificarFault(doc);
String nome = SoapXml.valor(doc, "nome");
```

O nome do elemento em `valor(...)` deve ser constante do seu código, nunca entrada do usuário (evita XPath injection).

11.5 Gerar classes Java a partir do WSDL

```
<!-- pom.xml: build/plugins — gera classes JAXB em target/generated-sources -->
<plugin>
  <groupId>org.jvnet.jaxb</groupId>
  <artifactId>jaxb-maven-plugin</artifactId>
  <version>4.0.8</version>
  <executions>
    <execution><goals><goal>generate</goal></goals></execution>
  </executions>
  <configuration>
    <schemaLanguage>WSDL</schemaLanguage>
    <schemaDirectory>src/main/resources/wsdl</schemaDirectory>
    <schemaIncludes><include>cadastro.wsdl</include></schemaIncludes>
    <generatePackage>com.exemplo.integracao.soap.cadastro</generatePackage>
  </configuration>
</plugin>
```

Salve o WSDL (e XSDs importados) no projeto em vez de baixar no build. Rode `./mvnw generate-sources` e confira as classes geradas. Alternativas: `cxf-codegen-plugin` (Apache CXF) ou `jaxws-maven-plugin`, se o projeto já usa JAX-WS.

11.6 Cliente tipado com Spring WS

```
@Configuration
public class SoapConfig {

  @Bean
  Jaxb2Marshaller cadastroMarshaller() {
    var m = new Jaxb2Marshaller();
    m.setContextPath("com.exemplo.integracao.soap.cadastro");   // pacote gerado
    return m;
  }

  @Bean
  WebServiceTemplate cadastroWs(Jaxb2Marshaller marshaller, CadastroProperties p) {
    var sender = new HttpUrlConnectionMessageSender();
    sender.setConnectionTimeout(Duration.ofSeconds(5));
    sender.setReadTimeout(Duration.ofSeconds(30));
```

```
    var ws = new WebServiceTemplate(marshaller);
    ws.setDefaultUri(p.url());
    ws.setMessageSender(sender);
    return ws;
  }
}

@Component
public class CadastroWsClient {
  private final WebServiceTemplate ws;
  public CadastroWsClient(WebServiceTemplate cadastroWs) { this.ws = cadastroWs; }

  public ConsultarClienteResponse consultar(String codigo) {
    var req = new ConsultarCliente();          // classe gerada do WSDL
    req.setCodigo(codigo);                     // JAXB faz o escape do XML
    try {
      return (ConsultarClienteResponse) ws.marshalSendAndReceive(req,
          new SoapActionCallback("ConsultarCliente"));
    } catch (SoapFaultClientException ex) {
      throw new IntegracaoException("SOAP Fault: " + ex.getFaultStringOrReason(), 500, ex);
    } catch (WebServiceIOException ex) {
      throw new IntegracaoException("Falha de conexão/timeout SOAP", 0, ex);
    }
  }
}
```

Para SOAP 1.2 com Spring WS: crie um `SaajSoapMessageFactory`, chame `setSoapVersion(SoapVersion.SOAP_12)` e passe ao `WebServiceTemplate`. Se a classe gerada for um `JAXBElement`, desembrulhe com `.getValue()`.

Abordagem	Quando usar
XML manual + RestClient (11.3)	1–2 operações, payload pequeno, integração pontual
Spring WS + JAXB (11.5–11.6)	Contrato extenso, muitos tipos, WS-Security, manutenção longa
Apache CXF	Já adotado na empresa, ou precisa de WS-* avançado

↑ Sumário

12. Testes automatizados

Teste cada camada isolada e simule o parceiro; o pipeline nunca deve depender de hml ou produção do fornecedor.

Camada	Ferramenta	Valida
Service	JUnit + Mockito	Regras de negócio, sem Spring
Controller	`@WebMvcTest` + MockMvc	Rotas, status, validação, JSON, handler de erros
Client REST	`MockRestServiceServer`	URL, método, headers, corpo enviado e parsing da resposta
Client REST ponta a ponta	WireMock	Timeout, retry, 5xx reais sobre HTTP

Camada	Ferramenta	Valida
Client SOAP	`MockWebServiceServer` (spring-ws-test)	Envelope enviado e resposta/Fault

12.1 Controller com MockMvc

```
@WebMvcTest(ClienteController.class)
class ClienteControllerTest {
  @Autowired MockMvc mvc;
  @MockitoBean ClienteService service;      // Boot < 3.4: @MockBean

  @Test
  void buscar_retorna200() throws Exception {
    when(service.buscar(10L))
        .thenReturn(new ClienteResponse(10L, "Ana", "ana@exemplo.com", Instant.EPOCH));

    mvc.perform(get("/api/v1/clientes/10"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.nome").value("Ana"));
  }

  @Test
  void criar_comEmailInvalido_retorna400() throws Exception {
    mvc.perform(post("/api/v1/clientes")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"nome":"Ana","email":"invalido"}
                """))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.campos[0].campo").value("email"));
  }

  @Test
  void buscar_inexistente_retorna404() throws Exception {
    when(service.buscar(99L)).thenThrow(new NaoEncontradoException("não encontrado"));
    mvc.perform(get("/api/v1/clientes/99")).andExpect(status().isNotFound());
  }
}
```

[Boot 4] as anotações de teste mudaram de pacote (ex.: `@WebMvcTest` em `org.springframework.boot.webmvc.test.autoconfigure`) e exigem o starter de teste do módulo; deixe a IDE resolver o import.

12.2 Client REST com MockRestServiceServer (sem subir Spring)

```
class ParceiroClientTest {
  MockRestServiceServer server;
  ParceiroClient client;

  @BeforeEach
  void setUp() {
    var builder = RestClient.builder().baseUrl("https://parceiro.test");
    server = MockRestServiceServer.bindTo(builder).build();
    client = new ParceiroClient(builder.build());
  }
```

```
  @Test
  void buscar_enviaGetEConverteJson() {
    server.expect(requestTo("https://parceiro.test/clientes/10"))
        .andExpect(method(HttpMethod.GET))
        .andRespond(withSuccess("""
            {"id":10,"nome":"Ana","email":"ana@exemplo.com","criadoEm":"2026-11-09T15:00:00Z"}
            """, MediaType.APPLICATION_JSON));

    var r = client.buscar(10L);

    assertThat(r.nome()).isEqualTo("Ana");
    server.verify();
  }

  @Test
  void criar_enviaCorpoEHeader() {
    server.expect(requestTo("https://parceiro.test/clientes"))
        .andExpect(method(HttpMethod.POST))
        .andExpect(header("X-Correlation-ID", "abc"))
        .andExpect(jsonPath("$.nome").value("Ana"))
        .andRespond(withCreatedEntity(URI.create("/clientes/1"))
            .contentType(MediaType.APPLICATION_JSON)
            .body("{\"id\":1,\"nome\":\"Ana\"}"));

    client.criar(new ClienteRequest("Ana", "ana@exemplo.com", null), "abc");
    server.verify();
  }

  @Test
  void erro500_viraExcecao() {

server.expect(requestTo("https://parceiro.test/clientes/10")).andRespond(withServerError());
    assertThatThrownBy(() ->
client.buscar(10L)).isInstanceOf(RestClientResponseException.class);
  }
}
```

12.3 Timeout real com WireMock

```
<dependency>
  <groupId>org.wiremock</groupId>
  <artifactId>wiremock-standalone</artifactId>
  <version>3.9.2</version>
  <scope>test</scope>
</dependency>
```

```
@SpringBootTest
class ParceiroClientWireMockTest {
  @RegisterExtension
  static WireMockExtension wm = WireMockExtension.newInstance()
      .options(wireMockConfig().dynamicPort()).build();

  @DynamicPropertySource
  static void props(DynamicPropertyRegistry r) {
    r.add("integracao.parceiro.url", wm::baseUrl);
```

```
    r.add("integracao.parceiro.token", () -> "token-teste");
    r.add("integracao.parceiro.read-timeout", () -> "1s");
  }

  @Autowired ParceiroClient client;

  @Test
  void respostaLenta_disparaTimeout() {
    wm.stubFor(get("/clientes/10").willReturn(okJson("{}").withFixedDelay(3_000)));
    assertThatThrownBy(() -> client.buscar(10L)).isInstanceOf(ResourceAccessException.class);
  }

  @Test
  void enviaBearer() {
    wm.stubFor(get("/clientes/10").willReturn(okJson("{\"id\":10,\"nome\":\"Ana\"}")));
    client.buscar(10L);
    wm.verify(getRequestedFor(urlEqualTo("/clientes/10"))
        .withHeader("Authorization", equalTo("Bearer token-teste")));
  }
}
```

Cuidado com nomes iguais: `get(...)` do WireMock e do MockMvc conflitam no mesmo arquivo; use import qualificado (`WireMock.get`).

↑ Sumário

13. Observabilidade e diagnóstico

Cada requisição precisa de um identificador que apareça em todos os logs e siga para o parceiro. Com isso, um chamado "deu erro às 14h" vira uma busca por ID.

O que observar	Como
Correlation ID	Filtro + MDC + interceptor (13.1)
Log estruturado	Evento, sistema, operação, status, duração (13.2)
Health check	`/actuator/health` com liveness/readiness
Métricas	`http.client.requests` e `http.server.requests` do Micrometer (latência, erros por status)
Tracing distribuído	`micrometer-tracing-bridge-otel` + exportador; propaga `traceparent` sozinho no RestClient injetado

13.1 Correlation ID de ponta a ponta

```
// Entrada: lê ou gera o ID e coloca no MDC (aparece em todo log da requisição)
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class CorrelationIdFilter extends OncePerRequestFilter {
  public static final String HEADER = "X-Correlation-ID";

  @Override
  protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                  FilterChain chain) throws ServletException, IOException {
    String id = Optional.ofNullable(req.getHeader(HEADER))
```

```
        .filter(v -> v.matches("[A-Za-z0-9-]{1,64}"))      // não confie no valor recebido
        .orElseGet(() -> UUID.randomUUID().toString());
    MDC.put("correlationId", id);
    res.setHeader(HEADER, id);
    try {
      chain.doFilter(req, res);
    } finally {
      MDC.remove("correlationId");
    }
  }
}

// Saída: repassa o ID ao parceiro
class CorrelationIdInterceptor implements ClientHttpRequestInterceptor {
  @Override
  public ClientHttpResponse intercept(HttpRequest req, byte[] body,
                                      ClientHttpRequestExecution exec) throws IOException {
    String id = MDC.get("correlationId");
    if (id != null) req.getHeaders().set(CorrelationIdFilter.HEADER, id);
    return exec.execute(req, body);
  }
}
// no builder do RestClient: .requestInterceptor(new CorrelationIdInterceptor())
```

MDC é por thread: em `@Async`, `CompletableFuture` ou virtual threads, copie o valor manualmente ou use um `TaskDecorator`.

13.2 Logs e Actuator

```
logging:
  pattern:
    level: "%5p [%X{correlationId:-}]"     # ID em toda linha de log
  structured:
    format:
      console: ecs                         # Boot 3.4+: JSON (ecs, logstash, gelf)

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus   # nunca "*" em produção
  endpoint:
    health:
      probes:
        enabled: true                      # /actuator/health/liveness e /readiness
      show-details: when-authorized
```

```
// Modelo de log de integração: o que, com quem, resultado, quanto tempo
log.info("integracao=parceiro operacao=buscarCliente status={} duracaoMs={} id={}",
         status, duracao, id);          // sem token, CPF completo ou payload sensível
```

13.3 Comandos de diagnóstico

Objetivo	Linux/macOS	Windows (PowerShell)
Health da aplicação	`curl -i http://localhost:8080/actuator/health`	`curl.exe -i http://localhost:8080/actuator/health`
DNS do parceiro	`nslookup api.exemplo.com`	`Resolve-DnsName api.exemplo.com`
Porta alcançável	`nc -vz api.exemplo.com 443`	`Test-NetConnection api.exemplo.com -Port 443`
TLS e certificado	`curl -Iv https://api.exemplo.com`	`curl.exe -Iv https://api.exemplo.com`
Cadeia de certificados	`openssl s_client -connect api.exemplo.com:443 -showcerts`	idem (Git Bash)
Quem usa a porta 8080	`ss -lntp | grep 8080`	`netstat -ano | findstr :8080`
Debug TLS na JVM	`java -Djavax.net.debug=ssl:handshake -jar app.jar`	idem

↑ Sumário

14. Segurança e checklist de publicação

Integrações costumam trafegar dados pessoais e de saúde: trate cada item abaixo como requisito, não como sugestão (LGPD).

Risco	Como se proteger
Segredo vazado no Git	Variável de ambiente ou cofre (Vault, Azure Key Vault, AWS Secrets Manager); `.env` no `.gitignore`; rotação periódica
Token ou dado pessoal em log	Nunca logue `Authorization`, senhas, CPF completo, prontuário ou XML/JSON integral; mascare (`***.456.789-**`)
Tráfego interceptado	HTTPS sempre; nunca "trust all certificates"; truststore com a cadeia correta
Entrada maliciosa	`@Valid` em todo DTO; limite de tamanho (`@Size`, `server.tomcat.max-swallow-size`, `spring.servlet.multipart.max-file-size`)
XML Injection / XXE	Escape de valores (JAXB ou `xml()` da 11.3); `disallow-doctype-decl` ao ler XML de terceiros
SSRF	URL do parceiro vem da configuração, nunca do usuário; se precisar, lista de hosts permitidos
Vazamento em erro	`ProblemDetail` genérico para o consumidor; detalhe só no log interno (seção 5)
Endpoint exposto sem proteção	Autenticação/autorização em toda rota; Actuator só com `health` público
Abuso / sobrecarga	Rate limit no gateway, `max-page-size`, timeouts em toda chamada externa

Checklist antes de publicar

- [ ] Contratos versionados (`/api/v1`), DTOs validados e documentação OpenAPI/WSDL com exemplos

- [ ] Segredos fora do código e dos logs; HTTPS e permissões mínimas nas credenciais do parceiro
- [ ] Connect e read timeout definidos em todos os clientes
- [ ] Política de retry documentada; POSTs que criam ou movimentam são idempotentes
- [ ] Erros externos mapeados para 502/504 sem vazar detalhes
- [ ] Testes de Service, Controller e Client passando no pipeline sem depender do parceiro
- [ ] Correlation ID, logs estruturados, métricas e alertas configurados
- [ ] Health check (liveness/readiness) ligado ao orquestrador
- [ ] URLs e credenciais separadas por ambiente (dev, hml, prod)
- [ ] Plano de rollback e contatos do parceiro (suporte, janela de manutenção) registrados

Para documentar sua própria API, adicione `org.springdoc:springdoc-openapi-starter-webmvc-ui` (versão compatível com seu Boot): a UI fica em `/swagger-ui.html` e o contrato em `/v3/api-docs`.

↑ Sumário

15. Colas rápidas

15.1 Anotações Spring

Anotação	Uso
`@RestController` / `@RequestMapping`	Expõe endpoints / prefixo de rota
`@GetMapping` `@PostMapping` `@PutMapping` `@PatchMapping` `@DeleteMapping`	Verbo HTTP do método
`@PathVariable` / `@RequestParam`	Parâmetro de rota / query string
`@RequestBody` / `@RequestHeader`	Corpo JSON / cabeçalho
`@ResponseStatus(HttpStatus.NO_CONTENT)`	Status fixo de sucesso
`@Valid` / `@Validated`	Dispara validação no DTO / em properties e serviços
`@NotBlank` `@NotNull` `@Email` `@Size` `@Pattern` `@Positive`	Regras de validação (jakarta.validation)
`@PageableDefault`	Tamanho e ordenação padrão da paginação
`@Service` / `@Component` / `@Repository`	Negócio / componente genérico / persistência
`@Configuration` + `@Bean`	Declara beans (RestClient, marshaller)
`@ConfigurationProperties` + `@ConfigurationPropertiesScan`	Propriedades tipadas
`@Qualifier("nome")`	Escolhe entre beans do mesmo tipo
`@Transactional(readOnly = true)`	Transação de banco
`@RestControllerAdvice` + `@ExceptionHandler`	Tratamento global de erros
`@HttpExchange` `@GetExchange` `@PostExchange`	Cliente REST declarativo
`@Retry` `@CircuitBreaker` (Resilience4j) / `@Retryable` (Spring 7)	Resiliência
`@WebMvcTest` `@SpringBootTest` `@MockitoBean`	Testes

15.2 Status HTTP

Código	Significado	Quando sua API devolve
200	OK	GET, PUT ou PATCH com corpo
201	Created	POST que criou recurso (+ header `Location`)
202	Accepted	Processamento assíncrono iniciado
204	No Content	DELETE ou PUT sem corpo
400	Bad Request	JSON malformado, validação falhou
401	Unauthorized	Sem credencial ou credencial inválida
403	Forbidden	Autenticado, mas sem permissão
404	Not Found	Recurso inexistente
405	Method Not Allowed	Verbo não suportado na rota
409	Conflict	Duplicidade, versão desatualizada
415	Unsupported Media Type	`Content-Type` errado
422	Unprocessable Content	Dados válidos no formato, inválidos na regra
429	Too Many Requests	Limite excedido (+ `Retry-After`)
500	Internal Server Error	Erro não tratado
502	Bad Gateway	Parceiro respondeu com erro
503	Service Unavailable	Sua API ou dependência fora (circuit aberto)
504	Gateway Timeout	Parceiro não respondeu a tempo

15.3 cURL

```
# GET com headers de resposta e detalhes da conexão
curl -i -v "http://localhost:8080/api/v1/clientes/10"

# GET com token e paginação
curl -H "Authorization: Bearer $TOKEN" "https://api.exemplo.com/clientes?page=0&size=20"

# POST JSON
curl -X POST "http://localhost:8080/api/v1/clientes" \
  -H "Content-Type: application/json" \
  -d '{"nome":"Ana","email":"ana@exemplo.com"}'

# PUT e DELETE
curl -X PUT "http://localhost:8080/api/v1/clientes/10" -H "Content-Type: application/json" -d
@cliente.json
curl -X DELETE -i "http://localhost:8080/api/v1/clientes/10"

# Basic auth
curl -u usuario:senha "https://api.exemplo.com/dados"

# OAuth2 client credentials: obter token
curl -X POST "https://auth.exemplo.com/oauth2/token" \
  -d "grant_type=client_credentials&client_id=$ID&client_secret=$SECRET&scope=clientes.read"

# SOAP 1.1 com envelope em arquivo
```

```
curl -X POST "https://soap.exemplo.com/servico" \
  -H "Content-Type: text/xml; charset=utf-8" \
  -H 'SOAPAction: "ConsultarCliente"' \
  --data-binary @requisicao.xml

# SOAP 1.2
curl -X POST "https://soap.exemplo.com/servico" \
  -H 'Content-Type: application/soap+xml; charset=utf-8; action="ConsultarCliente"' \
  --data-binary @requisicao12.xml

# Medir tempo de resposta
curl -o /dev/null -s -w "dns=%{time_namelookup} conn=%{time_connect} tls=%{time_appconnect}
total=%{time_total}\n" https://api.exemplo.com
```

No PowerShell use `curl.exe` (não o alias `curl`), troque `\` por `` ` `` no fim da linha e prefira aspas duplas por fora com `\"` por dentro, ou passe o corpo em arquivo com `-d @arquivo.json`.

↑ Sumário

16. Diagnóstico de problemas frequentes

Reproduza primeiro com cURL fora da aplicação: se falhar lá também, o problema é rede, credencial ou contrato, não o seu código.

Sintoma / exceção	Causa mais provável	Verifique
`401 Unauthorized`	Token expirado ou ausente	Header `Authorization` chegando, expiração, audience, relógio do servidor (NTP)
`403 Forbidden`	Credencial sem permissão	Scopes/roles do client, IP liberado no parceiro
`404 Not Found`	URL montada errada	`baseUrl` + path (barra duplicada ou faltando), versão da API, context path
`405 Method Not Allowed`	Verbo errado	GET x POST no contrato
`415 Unsupported Media Type`	`Content-Type` errado	JSON: `application/json`; SOAP 1.1: `text/xml`; SOAP 1.2: `application/soap+xml`
`400` sem detalhe	Corpo diferente do esperado	Nomes de campos (camelCase x snake_case), datas, campos obrigatórios; logue o corpo em dev
`429 Too Many Requests`	Limite do parceiro	Header `Retry-After`; reduza paralelismo
`ResourceAccessException: Read timed out`	Parceiro lento	Read timeout, SLA do parceiro, tamanho da página
`ConnectException` / `Connect timed out`	Rede bloqueada	DNS, firewall, proxy corporativo (`-Dhttps.proxyHost`), porta
`UnknownHostException`	DNS	`nslookup`, URL digitada, DNS interno x externo
`SSLHandshakeException: PKIX path building failed`	CA não confiável na JVM	Importe a cadeia no truststore (`keytool -importcert`); nunca desligue a validação

Sintoma / exceção	Causa mais provável	Verifique
`SSLHandshakeException: No subject alternative names`	Hostname não bate com o certificado	Use o hostname do certificado, não o IP
`HttpMessageNotReadableException` / JSON não desserializa	Contrato divergente	Tipos e nomes dos campos, formato de data, `FAIL_ON_UNKNOWN_PROPERTIES`
`No qualifying bean of type RestClient` / `expected single matching bean`	Dois RestClients sem qualificador	`@Qualifier` no ponto de injeção (seção 6.1)
`Could not resolve placeholder`	Propriedade ausente	Variável de ambiente, perfil ativo, `.env` no diretório de execução
`SOAP Fault: Client` / `Sender`	Requisição inválida	SOAPAction, namespace, nome da operação, elementos obrigatórios do XSD
`SOAP Fault: Server` / `Receiver`	Erro no parceiro	Abra chamado com correlation ID, horário e envelope enviado (sem dados sensíveis)
HTTP 500 com XML no corpo	SOAP Fault devolvido como 500	Leia o corpo antes de lançar erro (seção 11.3)
`502` / `503` vindo do parceiro	Upstream fora ou gateway	Status page do parceiro, circuit breaker, health
Retry/circuit breaker não funciona	Chamada interna na mesma classe ou falta AOP	Chamar via outro bean; dependência `spring-boot-starter-aop`
Caracteres trocados (Ã©)	Charset	`charset=UTF-8` no Content-Type e na leitura do corpo

Roteiro em 5 passos: (1) cURL fora da aplicação; (2) DNS e porta; (3) TLS com `curl -v`; (4) comparar requisição do código com a do cURL (URL, headers, corpo); (5) buscar o correlation ID nos logs dos dois lados.

↑ Sumário

17. Histórico de versões

v2.1 (nov/2026)

Navegação

- Sumário com links para todas as seções, atalhos para os casos mais usados e índice completo recolhível
- Link ↑ **Sumário** ao fim de cada seção
- Diagrama da arquitetura (seção 1) em Mermaid, renderizado pelo GitHub (antes era um placeholder quebrado)

Conteúdo novo: seção R

- Anatomia da URL: o que vai no `application.yml` e o que vai no código, com erros comuns (R1)

- REST sem autenticação com APIs públicas reais: ViaCEP (GET) e JSONPlaceholder (POST) (R2)
- REST com Basic, Bearer, API Key (header e query) e OAuth2, cada um com cURL + Java (R3–R6)
- SOAP sem autenticação com serviço público real (NumberConversion) (R7)
- SOAP com Basic no HTTP (RestClient e Spring WS) e com WS-Security UsernameToken (R8–R9)
- Tabela-resumo de onde vai a credencial em cada cenário (R10)

Correções

Na v2	Problema	Na v2.1
`fallbackMethod` no `@CircuitBreaker` junto com `@Retry`	O fallback trocava a exceção antes do Retry: nenhuma nova tentativa acontecia	`fallbackMethod` no `@Retry` (9.1)
`@Retryable(maxAttempts = 3, ...)` do Spring 7	Atributo não existe no Spring Framework 7	`maxRetries = 2` (= 3 chamadas), com explicação (9.2)
Exemplo 9.2 sem o campo `client`	Não compilava	Campo e construtor incluídos
Registro Boot 4 sem propriedades	Não mostrava como definir URL/timeout	`spring.http.serviceclient.<grupo>.*` (7.3)
`.env`: "só se houver o placeholder"	Omitia o binding relaxado	`INTEGRACAO_PARCEIRO_URL` explicado (3.3)
`oauth2ResourceServer` sem aviso	Falha na subida sem o starter/issuer	Dependência e propriedade indicadas (8.2)

v2 (v1 → v2)

A v2 corrige erros de código da v1, unifica a numeração (a v1 reiniciava a contagem de páginas na seção 23) e acrescenta os modelos que faltavam para o dia a dia.

Correções

Na v1	Problema	Na v2
`.header("SOAPAction", ""ConsultarCliente"")`	Não compila em Java	`"\"ConsultarCliente\""` (11.3)
`Bearer <token>` na tabela de autenticação	Entidade HTML impressa literalmente	`Bearer <token>` (8)
`RestClient.builder()` estático no exemplo Basic	Perde Jackson customizado, métricas e tracing	`RestClient.Builder` injetado (6.1, 8.1)
Dois beans `RestClient` injetados pelo nome do parâmetro	Ambiguidade ao adicionar outro parceiro	`@Qualifier` explícito (6.1)
`ClienteDto` definido em duas seções com usos diferentes	Confusão entre DTO interno e externo	`ClienteRequest`/`ClienteResponse` e `PaginaClientes` separados
Loop de páginas sem limite e sem tratar lista vazia	Risco de loop infinito	Limite máximo e saída por lista vazia (10)

Na v1	Problema	Na v2
Retornar `Page<T>` direto	JSON instável entre versões	`PaginaResponse<T>` ou `VIA_DTO` (4)
Erro genérico `Map.of("erro", ...)` com 502 sempre	Sem padrão e sem distinguir timeout	`ProblemDetail` com 502 x 504 (5)
Envelope SOAP montado com concatenação	Risco de XML injection	Escape `xml()` ou JAXB (11.3, 11.6)
Aviso de XXE sem código	Não dizia como desligar	Parser seguro pronto (11.4)

Conteúdo novo

- Pom.xml comentado e comandos Windows/Linux (2)
- `@ConfigurationProperties` tipado, perfis dev/prod e .env no PowerShell (3)
- CRUD completo com PUT, PATCH, DELETE, 201 + Location e limites de paginação (4)
- `exchange`, `onStatus`, `ParameterizedTypeReference`, interceptor de log (6)
- Cliente declarativo `@HttpExchange` (7)
- OAuth2 client credentials funcional, alerta de Spring Security e mTLS (8)
- Resilience4j configurado, `@Retryable` do Spring 7 e POST idempotente (9)
- Paginação por cursor (10)
- SOAP 1.1 x 1.2, mapa do WSDL, leitura de Fault, geração de classes e Spring WS (11)
- Testes de Client com MockRestServiceServer e WireMock (12)
- Correlation ID de ponta a ponta, log estruturado e diagnóstico em Windows (13)
- Tabela de riscos de segurança e diagnóstico por exceção com 20+ sintomas (14, 16)

Referências oficiais

- Spring Framework — REST Clients
- Spring Boot — Reference
- Spring Security — OAuth2 Client
- Spring Web Services — Reference
- Resilience4j — Spring Boot
- RFC 9110 — HTTP Semantics
- RFC 9457 — Problem Details
- OWASP — XXE Prevention

↑ Sumário